Distributed Scheduler - Part3 and Part4

we have seen how to achieve scheduling in single instance.

Now lets see the most common Pitfalls in Scheduling → and with this we ultimately design our distributed scheduler too

1. **Cron job + Time Zone issue:**

Problem:

```
@Scheduled(cron = "0 0 9 * * *")
public void myNotificationJob() {
    sendMorningUpdates();
}

```

Now, this cron expression means every day at "9 AM"

But, which time Zone?

By default, Cron picks Server time zone.

Concept && Coding

Server time zone : UTC

And

User time zone : IST

Cron job runs at 9:00 AM UTC = 2:30PM IST

User receives morning updates in afternoon.

Solution: Always specify Time zone

```
@Scheduled(cron = "0 0 9 * * *", zone = "Asia/Kolkata")
public void myNotificationJob() {
    sendMorningUpdates();

```

2. **Missing @Transactional**

Problem:

```
@Component
public class SubscriptionScheduler {

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

```

```
        List<Subscription> expired = subRepo.findByExpiryDateBefore(LocalDate.now());

        for (Subscription sub : expired) {

            // Step 1: Update subscription
            sub.setStatus("EXPIRED");
            subscriptionRepo.save(sub);

                // Step 2: Remove enrollment
            enrollmentRepo.deleteBySubscriptionId(sub.getId()); // throws Exception
             }
    }
}

```

Subscription #101:

Status update = EXPIRED (committed)

Enrollment deletion

Rollback is not happened for Subscription table status column, hence Data is now INCONSISTENT

<u>Solution:</u> Add @Transactional.

```
@Component
@Transactional
public class SubscriptionScheduler {

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

        List<Subscription> expired = subRepo.findByExpiryDateBefore(LocalDate.now());

        for (Subscription sub : expired) {

            // Step 1: Update subscription
            sub.setStatus("EXPIRED");
            subscriptionRepo.save(sub);

                // Step 2: Remove enrollment
            enrollmentRepo.deleteBySubscriptionId(sub.getId()); // throws Exception
             }
    }
}

```

Concept && Coding

Subscription #101:

Status update = ACTIVE (rolled back)

Enrollment deletion fails.

Data remains consistent. Next run will retry.

3. **<u>Fetching huge data - Memory Issue</u>**

<u>Problem:</u>

```
@Component
@Transactional
public class SubscriptionScheduler {
```

```java

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

        List<Subscription> expired = subRepo.findByExpiryDateBefore(LocalDate.now());

        for (Subscription sub : expired) {

            // Step 1: Update subscription
            sub.setStatus("EXPIRED");
            subscriptionRepo.save(sub);

                // Step 2: Remove enrollment
            enrollmentRepo.deleteBySubscriptionId(sub.getId());
             }
    }
}

```

what if this query fetches ***85000*** records

and

Say each record is for 10KB

So, all these records will be stored in memory and it will consume around:

85000 * 10KB = 850MB

Just for this query alone.

Assume, JVM heap: 512 MB allocated

Concept && Coding

then, this will result in

java.lang.OutOfMemoryError: Java heap space.

<u>Solution:</u> Batch processing

```java
@Component
@Transactional
public class SubscriptionScheduler {

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

        While(true) {

                        List<Subscription> expired = subRepo.fetchTop1000Expired(LocalDate.now());
                    If(expired.isEmpty()) {
                        break;
                    }

                    for (Subscription sub : expired) {

                            // Step 1: Update subscription
                            sub.setStatus("EXPIRED");
                            subscriptionRepo.save(sub);

                                // Step 2: Remove enrollment

enrollmentRepo.deleteBySubscriptionId(sub.getId());
 }
          }
      }
}
```

```

```

```
@Repository
public interface SubRepository extends JpaRepository<Subscription, Long> {

    @Query(value = """
        SELECT *
        FROM subscription
        WHERE expiry_date < :today
        AND status NOT IN ('EXPIRED')
        ORDER BY expiry_date ASC
        LIMIT 1000
    """, nativeQuery = true)
    List<Subscription> fetchTop1000Expired(@Param("today") LocalDate today);
}

```

Always double check the query filtering logic, as now we are within While(true), so it should not end up in infinite loop

This will fetch 1000 records at a time.

and this query will execute multiple times till there is record present in DB which satisfies the query.

Batch 1 -

→ Load 1000 records (10 MB)

→ GC can reclaim memory

Concept && Coding

Batch 2 -

→ Load 1000 records (10 MB)

→ GC can reclaim memory

.

.

Batch 85 -

→ Load 1000 records (10 MB)

→ GC can reclaim memory

4. **Memory Leak**

Even with above batch processing, Memory issue is not resolved.

Batch 1 -

→ Load 1000 records (10 MB)

→ GC can reclaim memory

We are saying, GC can reclaim memory after the batch run. But will it be able to reclaim?

No.

And because of it, with each batch run, Memory consumption grows.

Problem:

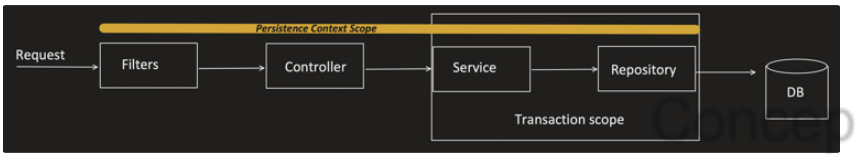
Everything is running under 1 big @Transactional.

So, JPA Persistence Context (First level Cache) keeps reference to all entities loaded during the transaction.

Note:

- HTTP Request → Open Session In View enabled by Default

  means, 1 Persistence Context per HTTP request



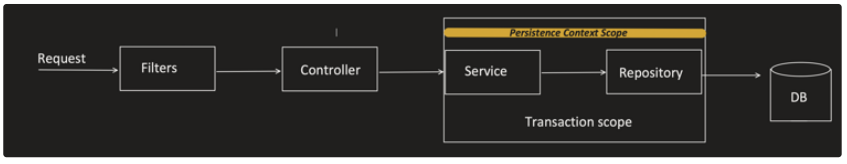
Concept && Coding

- HTTP Request → Open Session In View disabled

application.properties

spring.jpa.open-in-view = false

means, 1 Persistence Context per @Transactional



- For Scheduler / Background jobs → No HTTP request exist

So 1 Persistence Context per @Transactional is the only option.

```
@Component
@Transactional
public class SubscriptionScheduler {

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

```

```
        While(true) {

                        List<Subscription> expired = subRepo.fetchTop1000Expired(LocalDate.now());
                    If(expired.isEmpty()) {
                        break;
                    }

                    for (Subscription sub : expired) {

                            // Step 1: Update subscription
                            sub.setStatus("EXPIRED");
                            subscriptionRepo.save(sub);

                                // Step 2: Remove enrollment

enrollmentRepo.deleteBySubscriptionId(sub.getId());
 }
          }
      }
}

```

```
@Repository
public interface SubRepository extends JpaRepository<Subscription, Long> {

    @Query(value = """
        SELECT *
        FROM subscription
        WHERE expiry_date < :today
    AND status NOT IN ('EXPIRED')
        ORDER BY expiry_date ASC
        LIMIT 1000
    """, nativeQuery = true)
    List<Subscription> fetchTop1000Expired(@Param("today") LocalDate today);
}

```

Solution: We need @Transactional per Batch

Concept && Coding

```
@Component
public class SubscriptionScheduler {

@Autowired
private SubscriptionService subService;

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

        While(true) {

                    boolean processed = subService.processBatch();
                    If(!processed) {
                          break;
                    }
            }
      }
}

```

```
@Service
public class SubscriptionService {

   @Autowired
    private SubRepository subRepo;

   @Autowired
    private EnrollmentRepository enrollmentRepo;


    @Transactional
    public boolean processBatch() {

        List<Subscription> expired =

subRepo.fetchTop1000Expired(LocalDate.now());

        if (expired.isEmpty()) {
            return false;
        }

        for (Subscription sub : expired) {
```

```
            sub.setStatus("EXPIRED");
          subRepo.save(sub);

            enrollmentRepo.deleteBySubscriptionId(sub.getId());
        }
        return true;
    }
}

```

```
@Repository
public interface SubRepository extends JpaRepository<Subscription, Long> {

    @Query(value = """
        SELECT *
        FROM subscription
        WHERE expiry_date < :today
       AND status NOT IN ('EXPIRED')
        ORDER BY expiry_date ASC
        LIMIT 1000
    """, nativeQuery = true)
    List<Subscription> fetchTop1000Expired(@Param("today") LocalDate today);
}

```

5. **Exception handling**

Problem:

- 1 bad subscription, entire batch fails.
- 1 batch failure, whole scheduler fails.
- No logging

Handling: 1 bad subscription, entire batch fails scenario.

Concept && Coding

```
@Service
public class SubscriptionService {

   @Autowired
    private SubRepository subRepo;

   @Autowired
    private EnrollmentRepository enrollmentRepo;


    @Transactional
    public boolean processBatch() {

        List<Subscription> expired =
subRepo.fetchTop1000Expired(LocalDate.now());

        if (expired.isEmpty()) {
            return false;
        }

        for (Subscription sub : expired) {
            sub.setStatus("EXPIRED");
          subRepo.save(sub);
          /*
              Failure Scenario
            • Subscription #900th is in processing
            • And while deleting enrolment, it throws DB constraint error
            • Whole transaction rolls back
            • 899 already processed lost
            Remaining 100 never touche
            */

            enrollmentRepo.deleteBySubscriptionId(sub.getId());
        }
        return true;
    }
}

```

Improved Record Handling:

```java
@Service
public class SubscriptionService {

   @Autowired
    private SubRepository subRepo;

   @Autowired
    private EnrollmentRepository enrollmentRepo;


    @Transactional
    public boolean processBatch() {

        List<Subscription> expired =

subRepo.fetchTop1000Expired(LocalDate.now());

        if (expired.isEmpty()) {
            return false;
        }

        for (Subscription sub : expired) {
               try {
                    sub.setStatus("EXPIRED");
                  subRepo.save(sub);


enrollmentRepo.deleteBySubscriptionId(sub.getId());
  /*
    Failure Scenario handling:
    • Subscription #900th is in processing
    • And while deleting enrolment, it throws DB constraint error
    • Now bad transaction we are caching and handling it according
    to business requirement.
    • Remaining 100, continue processing.
   */
                } catch (Exception e) {
                     log.error("Failed sub {}", sub.getId(), e);
                     sub.incrementRetryCount();
                     sub.setStatus("FAILED");
                     subRepo.save(sub);
                }
      }
        return true;
    }
}

```

Concept && Coding

```java

@Repository
public interface SubRepository extends JpaRepository<Subscription, Long> {

    @Query(value = """
        SELECT *
        FROM subscription
        WHERE expiry_date < :today
       AND status NOT IN ('EXPIRED', 'FAILED')
        ORDER BY expiry_date ASC
        LIMIT 1000
    """, nativeQuery = true)
    List<Subscription> fetchTop1000Expired(@Param("today") LocalDate today);
}

```

Status update is made, to avoid infinite loop, as we are within while loop

Handling: 1 batch failure, whole scheduler fails

```java
@Component
public class SubscriptionScheduler {

@Autowired
private SubscriptionService subService;

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {
```

```

        While(true) {
        /*
        Failure Scenario
    - If say batch #3, while processing, throws exception : say while fetching
      from DB.
    • Scheduler stops entirely.
    • Batch #1 and #2 are safe (committed)
    • But no continuation for the remaining batches
       */
                    boolean processed = subService.processBatch();
                    If(!processed) {
                          break;
                    }
            }
      }
}

```

```
@Service
public class SubscriptionService {

   @Autowired
    private SubRepository subRepo;

   @Autowired
    private EnrollmentRepository enrollmentRepo;


    @Transactional
    public boolean processBatch() {
   /*
 Failure Scenario
    - If say batch #3, while processing, throws exception : say while fetching
    from DB.
    • Scheduler stops entirely.
    • Batch #1 and #2 are safe (committed)
    • But no continuation for the remaining batches
   */
        List<Subscription> expired =

subRepo.fetchTop1000Expired(LocalDate.now());

        if (expired.isEmpty()) {
            return false;
        }

        for (Subscription sub : expired) {
               try {
                    sub.setStatus("EXPIRED");
                  subRepo.save(sub);


enrollmentRepo.deleteBySubscriptionId(sub.getId());

                } catch (Exception e) {
                     log.error("Failed sub {}", sub.getId(), e);
                     sub.incrementRetryCount();
                     sub.setStatus("FAILED");
                     subRepo.save(sub);
                }
      }
        return true;
    }
}

```

Concept && Coding

Improved Batch Handling

```
@Component
public class SubscriptionScheduler {

@Autowired
private SubscriptionService subService;

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

        While(true) {

```

```
                    boolean processed = subService.processBatch();
                    If(!processed) {
                          break;
                    }
            }
      }
}

```

```
@Service
public class SubscriptionService {

   @Autowired
    private SubRepository subRepo;

   @Autowired
    private EnrollmentRepository enrollmentRepo;


    @Transactional
    public boolean processBatch() {

    try {

            List<Subscription> expired =
subRepo.fetchTop1000Expired(LocalDate.now());

            if (expired.isEmpty()) {
                return false;
            }

            for (Subscription sub : expired) {
                   try {
                        sub.setStatus("EXPIRED");
                      subRepo.save(sub);


enrollmentRepo.deleteBySubscriptionId(sub.getId());

                    } catch (Exception e) {
                         log.error("Failed sub {}", sub.getId(), e);
                         sub.incrementRetryCount();
                         sub.setStatus("FAILED");
                         subRepo.save(sub);
                    }
            }

        } catch(Exception e) {
        /*
        - If say batch #3, while processing, throws exception : say while
        fetching from DB.

        - Only this batch get impacted.

        - Other remaining batch can still continue
        */
               log.error("Batch DB failure", e);
        }
        return true;
    }
}

```

Concept && Coding

Handling: Logging, alert, protect entire job loop

```
@Component
public class SubscriptionScheduler {

@Autowired
private SubscriptionService subService;

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

        While(true) {
          /*
          Any uncaught exception, might silently kill our scheduler job
          */

                    boolean processed = subService.processBatch();
                    If(!processed) {
```

```
                        break;
                    }
            }
        }
}

```

Improved Overall scheduler job

```java
@Component
public class SubscriptionScheduler {

@Autowired
private SubscriptionService subService;

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

        try{

            While(true) {

                    boolean processed = subService.processBatch();
                    If(!processed) {
                          break;
                    }
                 }
        } catch(Exception e) {
                /*
                Just a Full proof protection that, there is no silent failures.
                */
                log.error("Scheduler failed", e);
                 alertService.notify(e);
        }
    }
}

```

Concept && Coding

6. **DB operation Batch Execution**

Use case:

Data

- Total expired = **85,000**
- Batch size = **1000**
- Batches = **85**

Per Record

- 1 UPDATE subscription
- 1 DELETE enrollment

So:

1000 records × 2 SQL = 2000 SQL per batch
85 batches × 2000 = 170,000 SQL statements

Generally:

- Hibernate may group some SQL
- But logically, say **~170k SQL executions,** will happen

Solution : Hibernate Batching

We still internally execute 17k sql executions,

But we will save DB network calls (less round trips due to batching)

application.properties

spring.jpa.properties.hibernate.jdbc.batch_size=50

spring.jpa.properties.hibernate.order_updates=true

Without "order_updates":

Say we are performing operation:

- Update subscription id = 1
- Update enrollment id = 5
- Update subscription id = 2
- Update enrollment id = 6

Because of table switch, batching will not work.

So,

Concept && Coding

With "order_updates":

We are allowing hibernate to re-arrange the operations:

- Update subscription id = 1
- Update subscription id = 2
- Update enrollment id = 5
- Update enrollment id = 6

Now batching can work

batch_size=50

mean, hibernate will collect up to 50 similar SQL statements in memory.

Now instead of sending 1 by 1, it send them together using 1 JDBC batch call to DB driver.

So network round trips are reduced drastically. Which result in:

- Lower CPU usage
- Shorter job duration

7. **Multiple instances : Duplication Execution**

Problem:

Microservice Instance 1 and Microservice Instance 2 both run the same job at 1am everyday

```
@Component
public class SubscriptionScheduler {

@Autowired
private SubscriptionService subService;

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

        try{

            While(true) {

                    boolean processed = subService.processBatch();
                    If(!processed) {
                          break;
                    }
                }
        } catch(Exception e) {

               log.error("Scheduler failed", e);
                alertService.notify(e);
        }
    }
}

```

What Happens Internally:

Step	Pod-1	Pod-2
1	SELECT top 1000	SELECT same top 1000
2	Update status	Update status again
3	Delete enrolment	Delete again / error

Concept && Coding

So, we are doing duplicate execution of same job, and if this is not handled properly, this can cause various problem like sending money multiple times, sending notification multiple times etc.

Solution1: PESSIMISTIC LOCK

```
@Component
public class SubscriptionScheduler {

@Autowired
private SubscriptionService subService;

    @Scheduled(cron = "0 0 1 * * *")
    public void expireSubscriptions() {

        try{

            While(true) {

                    boolean processed = subService.processBatch();
                    If(!processed) {
                          break;
                    }
                }
        } catch(Exception e) {

               log.error("Scheduler failed", e);
                alertService.notify(e);
        }
    }
}
```

```

```

```java
@Service
public class SubscriptionService {

   @Autowired
    private SubRepository subRepo;

   @Autowired
    private EnrollmentRepository enrollmentRepo;


    @Transactional
    public boolean processBatch() {

    try {

            List<Subscription> expired =

subRepo.fetchTop1000Expired(LocalDate.now());

            if (expired.isEmpty()) {
                return false;
            }

            for (Subscription sub : expired) {
                   try {
                        sub.setStatus("EXPIRED");
                      subRepo.save(sub);


enrollmentRepo.deleteBySubscriptionId(sub.getId());

                     } catch (Exception e) {
                          log.error("Failed sub {}", sub.getId(), e);
                          sub.incrementRetryCount();
                          sub.setStatus("FAILED");
                          subRepo.save(sub);
                     }
            }

        } catch(Exception e) {
               log.error("Batch DB failure", e);
        }
        return true;
    }
}

```

Concept && Coding

```java
@Repository
public interface SubRepository extends JpaRepository<Subscription, Long> {

    @Query(value = """
        SELECT *
        FROM subscription
        WHERE expiry_date < :today
       AND status NOT IN ('EXPIRED', 'FAILED')
        ORDER BY expiry_date ASC
        LIMIT 1000
           /*
           Lock will be acquired on 1000 rows by
            only 1 txn.
          Others txns will wait for lock to release.
           */
        FOR UPDATE
    """, nativeQuery = true)
    List<Subscription> fetchTop1000Expired(@Param("today") LocalDate today);
}

```

Runtime Behavior (2 instances)

Instance1

BEGIN TX

SELECT ... FOR UPDATE → locks 1000 rows

Process

COMMIT → unlock

Instance2

BEGIN TX

SELECT ... FOR UPDATE → BLOCKS

(waiting for instance1 commit)

Timeline Example

Time	Instance-1	Instance-2
01:00	Locks rows	Waiting
01:02	Processing	Waiting
01:03	Commit	Gets lock
01:03		Now processes

Cons (Important):

Blocking nature:

Its not scalable, as other instances are just blocked by 1 instance.
Like if Instance1 get blocked for 1 hr (long running tasks), all other instances also get blocked for 1hr.

Concept && Coding

Go for it only when:

- Instance are less (1 or 2)
- Small batch and strict order is required.

Can we resolve this issues?

Yes, if strict ordering is not required, then we can do parallel processing.

Solution2: PESSIMISTIC LOCK But with SKIP LOCKED feature

the **SKIP LOCKED** feature, allows a query to bypass rows already locked by other transactions.

Note: Skip locked feature availability depends on DB to DB and their respective versions. But all major DBs support this.

Instance-1 → take first 1000

Instance-2 → take next 1000

Instance-3 → next...

So, what we get:

- No duplicates.
- No waiting.
- Parallel processing.

Which is good for scalability.

```
@Repository
public interface SubRepository extends JpaRepository<Subscription, Long> {

    @Query(value = """
        SELECT *
        FROM subscription
        WHERE expiry_date < :today
       AND status NOT IN ('EXPIRED', 'FAILED')
        ORDER BY expiry_date ASC
        LIMIT 1000
        FOR UPDATE SKIP LOCKED
    """, nativeQuery = true)
    List<Subscription> fetchTop1000Expired(@Param("today") LocalDate today);
}

```

What if requirement is: <u>ONLY 1 INSTANCE SHOULD RUN THE JOB, How to achieve this?</u>

Answer is: **ShedLock**

Without ShedLock → All instance execute the Scheduler job. And with PESSIMISTIC LOCK, we put lock on rows to avoid duplicate execution.

Concept && Coding

With ShedLock → only 1 instance execute the Scheduler job, others simply ***SKIP*** the scheduler job execution if 1 instance is already executing it.

Before we implement ***ShedLock***, lets understand its functionality step by step:

1. New table is created to manage scheduler jobs:

"**<u>shedlock</u>**" table

(Primary key)

Name		lock_until	locked_by

Name: every Scheduler job, is given a unique name

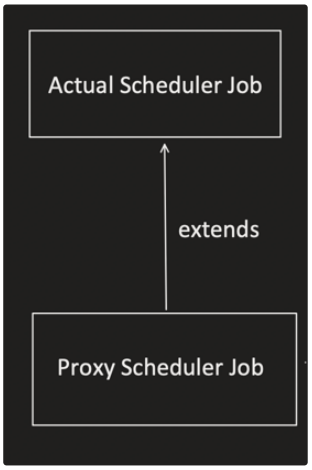
locked_at: consider it as Time, when some worker thread (in any one of the Pod) starts working on this job.

Why I did not say, *"time when someone puts a lock"* because in ShedLock no row is locked, its work more on OPTIMISTIC CONCURRENCY functionality.

locked_until: after this time, any scheduler instance(different pods) is allowed to acquire and execute the job again.

locked_by: identity of the instance/worker that last acquired the job lease.
in other words:
It shows who is currently running the job, or, if the job has finished and no new worker has taken it yet, who ran it last.

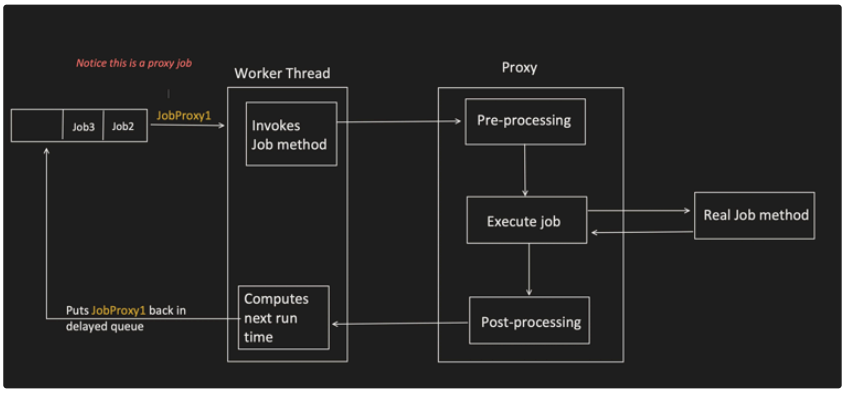
2. Interceptor is added, whenever job schedule time arrives, call goes to interceptor and it performs tasks before and after execution of the job.



Concept && Coding

```java
class JobProxy extends Job {

    @Override
    public void runMyJob() {

            //Pre-Processing

        super.runMyJob();  //actual job

            //Post-Processing
        }
    }
}

```

For which ever scheduler job, we want shedLock feature, in delayed queue, this Proxy job only get inserted.



Now question is, what tasks it does before execution of the job and after completion of the job

Lets understand in sequence:

Step1: insert the proxy job in delayed queue

When application startup happens, *@EnableScheduling* scans *@Scheduled* methods and register them (insert them in delayed queue)

```
@SpringBootApplication
@EnableScheduling
@EnableSchedulerLock(defaultLockAtMostFor="10m")
public class SchedulerApplication {

        public static void main(String[] args) {
            SpringApplication.run(SchedulerApplication.class, args);
        }
}

```

```
@Component
public class SchedulersTest {

@Scheduled(fixedRate = 300000)
@SchedulerLock(name="myJob", lockAtMostFor="8m", lockAtLeastFor="4m")
public void myJob() {
        System.out.println(" job execution started");
}

```

Note:

But when we are using ShedLock for particular scheduler job, it creates Proxy for that job first, and that proxy job is inserted into delayed queue.

Concept && Coding

***@SchedulerLock* takes parameter:**

- Name: (optional)
  unique name which you want to give to this scheduler job.
  Stored in DB.

If we don't give name, internally it generates one by combining ***class name*** and ***method name.***

- **lockAtMostFor: (optional)**

After this time, any scheduler instance(different pods) is allowed to acquire and execute the job again.

Stored in DB.

If we don't give here, it will use the default one (given in @EnableSchduelerLock annotation)

- **lockAtLeastFor: (optional)**

Even the instance which acquire the job, should not release it before this much time, even if job is finished early.

Not Stored in DB. Its just a configuration value.

@EnableSchedulerLock

@EnableSchedulerLock(defaultLockAtMostFor="10m")

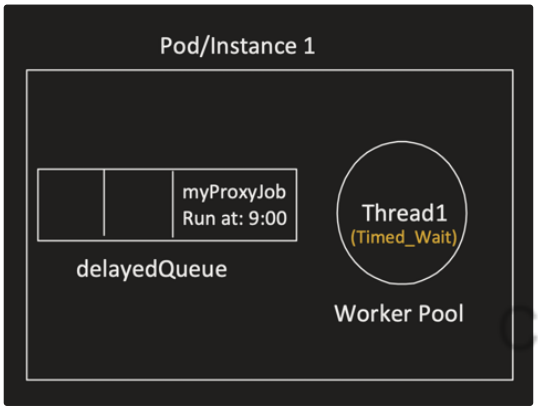
- It enables the shedLock functionality. In other words, It loads all the bean related to shedLock.
- It also scan the @schedulerlock methods and creates the proxy during application startup.

1st Dependency we need to add for this is:

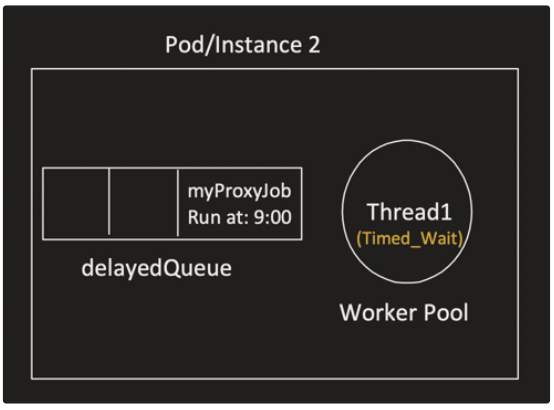
pom.xml

```
<dependency>
            <groupId>net.javacrumbs.shedlock</groupId>
            <artifactId>shedlock-spring</artifactId>
            <version>5.13.0</version>
  </dependency>

```

Current Time is 8:55



Concept && Coding



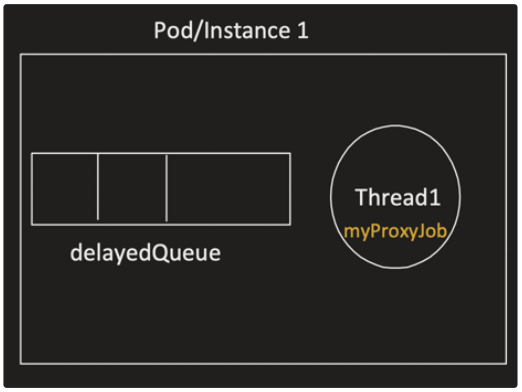
"**shedlock**"

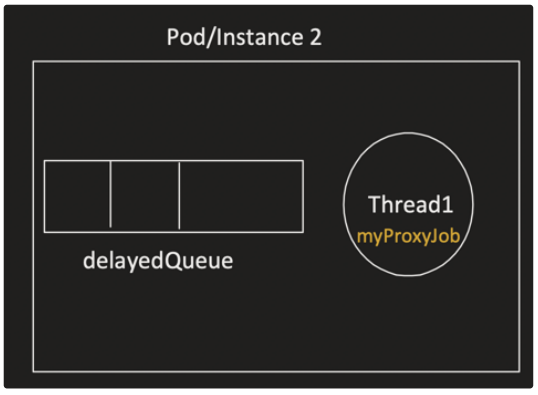
(Primary key)

Name	locked _at	lock_until	locked_by

Step2: pick the proxy job from delayed queue

Now lets say, Current Time is 9:00 and worker thread wakes up in both the Pods.



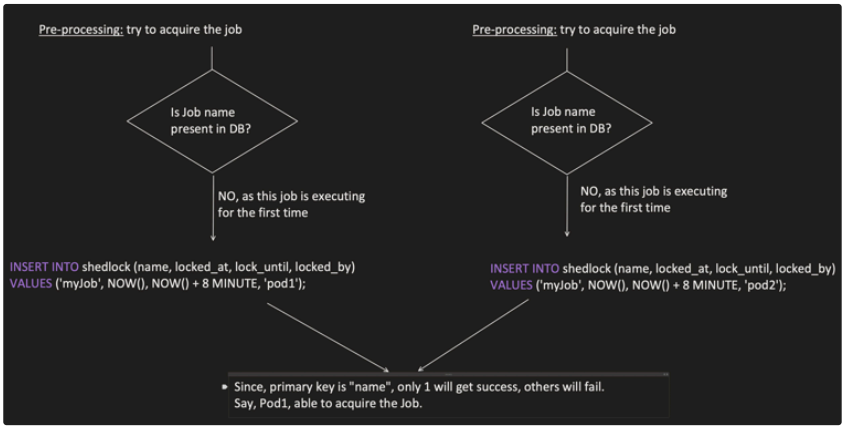


Concept && Coding

Run Proxy Job:

job configuration was:

- *Name: "myJob"*
- *lockAtMostFor=8m*
- lockAtLeastFor=4m
- *FixedRate = 5m*



"**shedlock**"

(Primary key)

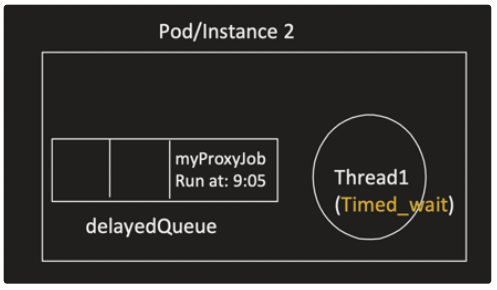
name	locked_at	lock_until	locked_by
myJob	9:00	9:08	pod1

Pod2 failed to acquire the job, so it will skip the execution.

Pod2, will compute the next run time for this job:

- for fixedRate (next run time is) = previous runtime + delay = 9:00 + 5m = 9:05

And put it back to its delayedQueue.



Lets stop here a bit, before we proceed further.

proxy creation, pre-processing etc. logic is all provided by shedLock library. We do not have to write those.

Concept && Coding

But I think we need to provide them is:

- Which storage to use:
  - MySQL
  - Redis
  - MongoDB

Etc.

Also, we do not have to worry about writing those INSERT , UPDATE (later will see) queries too. Because shedLock has wrapper libraries around all (MySQL, Redis etc...)

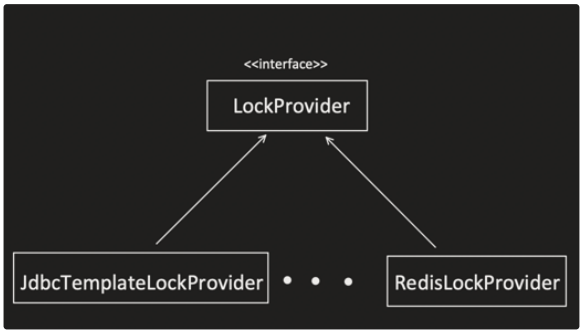
For example: I am using MySQL, so I have to add JDBC shedLock wrapper dependency. And create its bean.

2nd Dependency that we are adding is (if you are using Relational DB):

pom.xml

```
<dependency>
<groupId>net.javacrumbs.shedlock</groupId>
<artifactId>shedlock-provider-jdbc-template</artifactId>
<version>5.13.0</version>
</dependency>

```

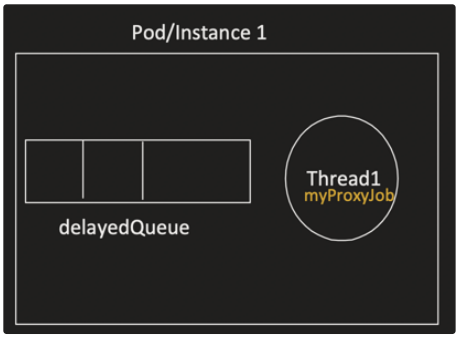
Apart from this, we need: DB and its driver dependency (like what we covered in JPA topics)

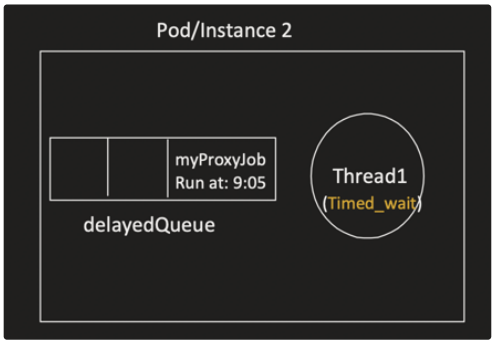



```java
@Configuration
public class ShedLockConfig {

    @Bean
  /*
      Datasource has all the info:
      db url, username, password etc.…
  */
    public LockProvider lockProvider(DataSource dataSource) {
        return new JdbcTemplateLockProvider(
                JdbcTemplateLockProvider.Configuration.builder()
                .withJdbcTemplate(new JdbcTemplate(dataSource))
                .usingDbTime() //safer for multi-pod
                .build()
                );
        }
}

```

Concept && Coding

<u>Step3:</u> Pod1 finishes the task before next run time itself say 9:03





Run Proxy Job:

job configuration was:

- *Name: "myJob"*

- *lockAtMostFor=8m*
- lockAtLeastFor=4m
- *FixedRate = 5m*

"**shedlock**"

(Primary key)

name	locked_at	lock_until	locked_by
myJob	9:00	9:08	pod1

Pre-processing: done

Invoke real job method: done and job finishes at 9:03

Post-processing:

Since task is finished early, we can update the ***"lock_until"*** column. So that, after this new time, this job is free to be picked by any other scheduler instance.

New Lock_Until column value = **max**(now() , locked_at + lockAtLeastFor)

Concept && Coding

now() = 9:03

locked_at + lockAtLeastFor = 9:00 + 4m = 9:04

newLockUntil = max(9:03, 9:04) = 9:04

UPDATE shedlock **SET** lock_until = :newLockUntil **WHERE** name = 'myJob' **AND** locked_by = 'pod1';

"**shedlock**"

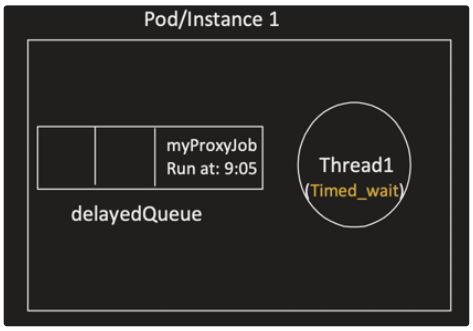
(Primary key)

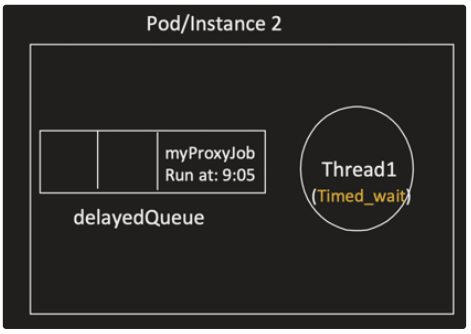
name	locked_at	lock_until	locked_by
myJob	9:00	9:04	pod1

Step4: Pod1 pushes the job back to its delayed queue with new run time

This finished job is inserted back to Pod1 delayed queue with next run time.

Fixed Delay next run time = previous runt time + delay = 9:00 + 5m = 9:05





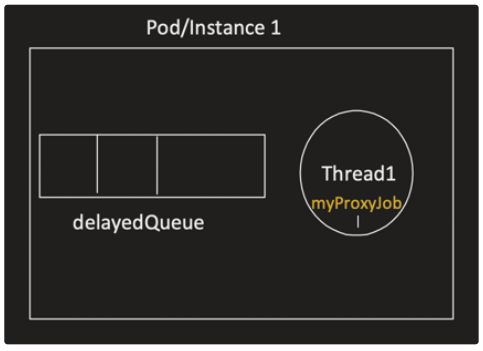
"**shedlock**"

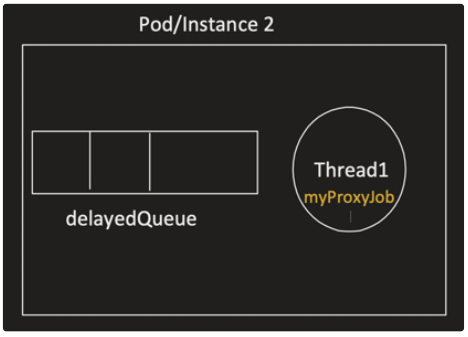
(Primary key)

name	locked_at	lock_until	locked_by
myJob	9:00	9:04	pod1

Concept && Coding

Step5: Next run time 9:05 arrives and both POD workers wakes up

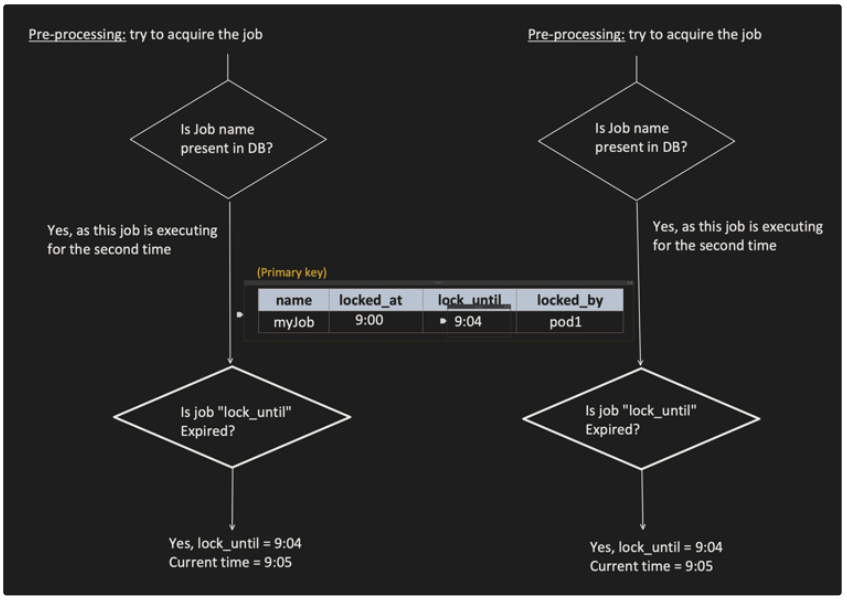


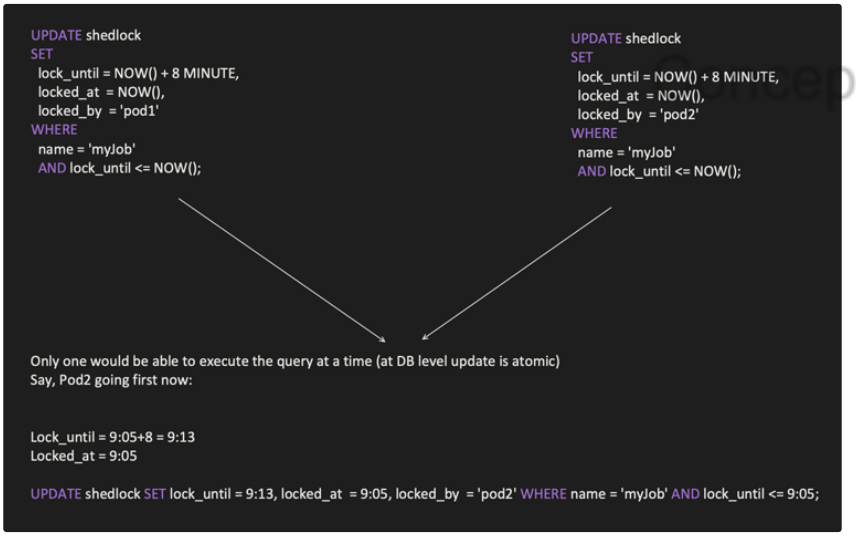


Run Proxy Job

job configuration was:

- *Name: "myJob"*
- *lockAtMostFor=8m*
- lockAtLeastFor=4m
- *FixedRate = 5m*





Concept && Coding

(Primary key)

name	locked_at	lock_until	locked_by
myJob	9:00	9:04	pod1

Now, Pod1 query get executed:

UPDATE shedlock SET lock_until = 9:13, locked_at = 9:05, locked_by = 'pod1' WHERE name = 'myJob' AND lock_until <= 9:05;

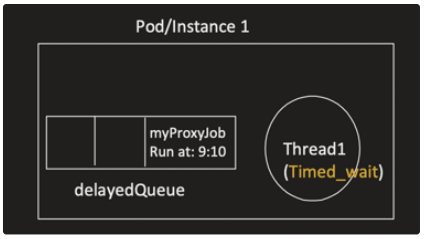
But this query will have no impact because ***"lock_until"*** is now more than 9:05, so where condition does not match any row.

Pod1 failed to acquire the job, so it will skip the execution.

Pod1, will compute the next run time for this job:

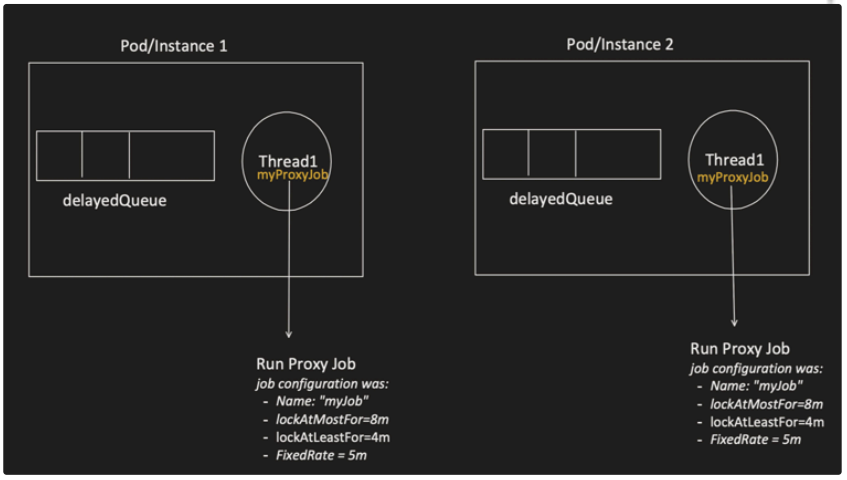
- for fixedRate (next run time is) = previous runtime + delay = 9:05 + 5m = 9:10

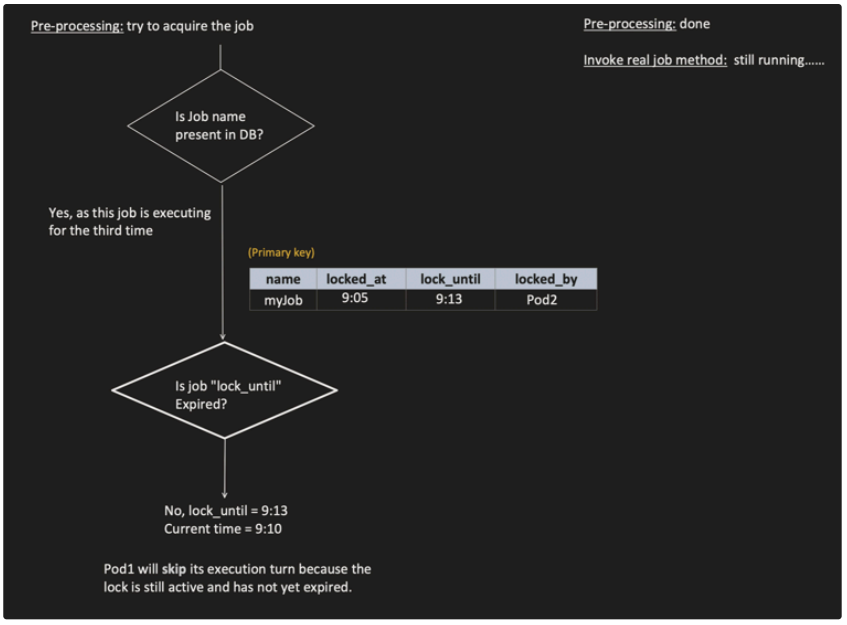
And put it back to its delayedQueue.



<u>Step6:</u> Pod2 is taking too long time while executing the job, so Pod2 is still running the job and next run time has come 9:10

Concept && Coding



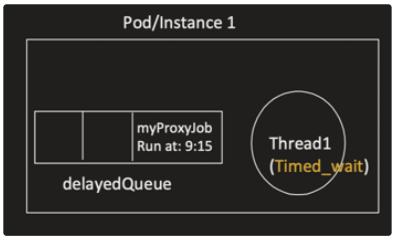


Pod1 failed to acquire the job, so it will skip the execution.

Pod1, will compute the next run time for this job:

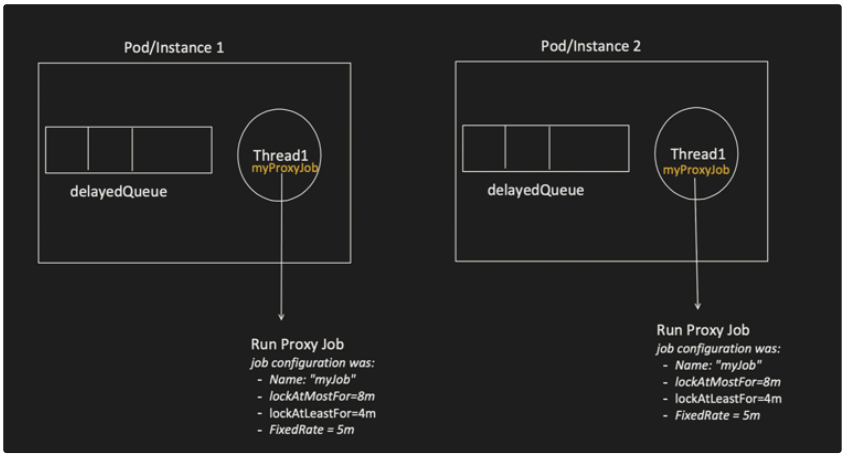
- for fixedRate (next run time is) = previous runtime + delay = 9:10 + 5m = 9:15

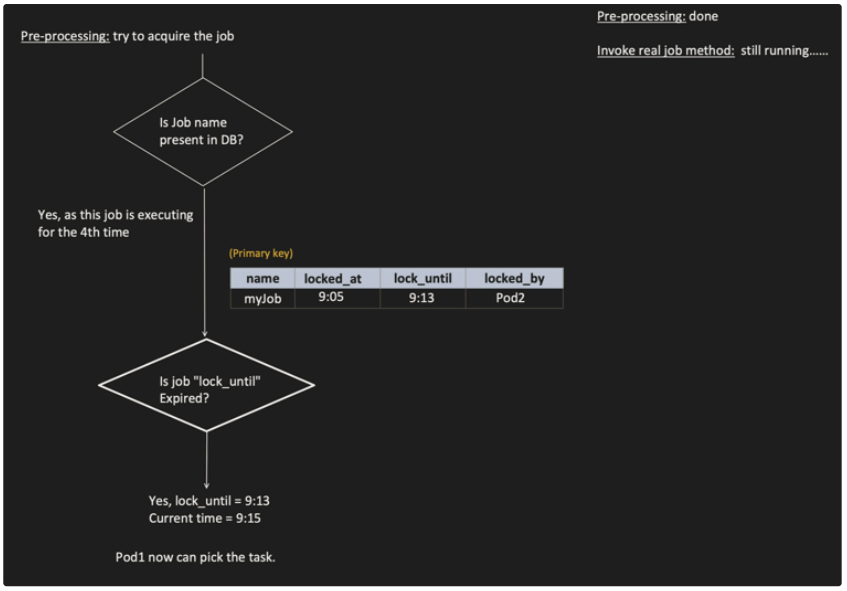
And put it back to its delayedQueue.



Concept && Coding

Step7: Pod2 is still taking too long time while executing the job, next execution time comes **9:15**, but Pod2 is still running the job.






```
UPDATE shedlock
SET
  lock_until = NOW() + 8 MINUTE,
  locked_at  = NOW(),
  locked_by  = 'pod1'
WHERE
  name = 'myJob'
  AND lock_until <= NOW();
```

Concept && Coding

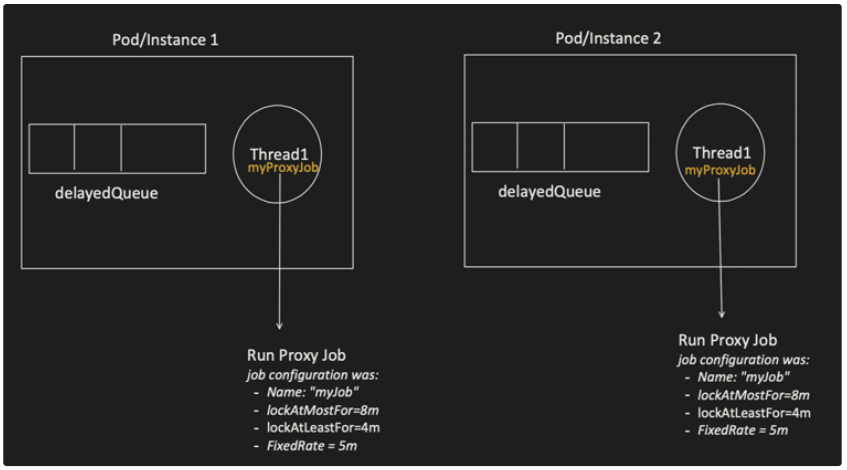
lock_until = 9:15 + 8m = 9:23

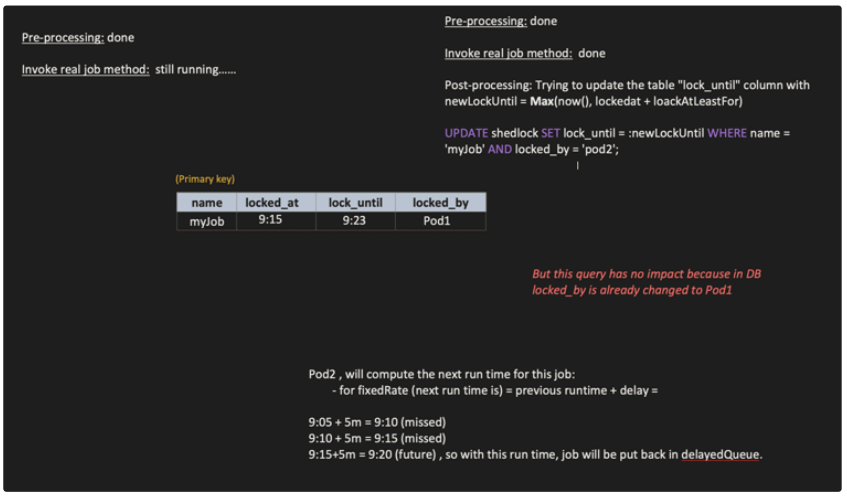
(Primary key)

name	locked_at	lock_until	locked_by
myJob	9:15	9:23	Pod1

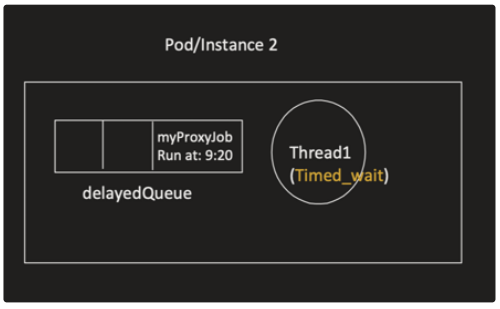
Invoke : the real job method

Step8: Pod1 is working on the job and now Pod2 finished say at 9:18





Concept && Coding



And it goes on.....

Last but not the least:

If you want custom table name and column name, then we have to update the bean too.

Generally default is:

- Table : **shedlock**
- jobname : **name**

- Expiry Column : **lock_until**
- Locked At Column : **locked_at**
- Locked By Column : **locked_by**

If we want different names for table and column, then we just need to tell, JdbcTemplateLockProvider too.

```
public LockProvider lockProvider(DataSource dataSource) {
return new JdbcTemplateLockProvider(
JdbcTemplateLockProvider.Configuration.builder()
.withJdbcTemplate(new JdbcTemplate(dataSource))
.withTableName("scheduler_lock")
.withColumnNames(
"job_name",
"expires_at",
"started_at",
"instance_id")
.usingDbTime()
.build()
);
}
```

Concept && Coding